Taaltheorie en Taalverwerking

BSc KI / Artificial Intelligence

2026 – Lecture 2

Katrin Schulz

k.schulz@uva.nl

Institute for Logic, Language, and Computation

University of Amsterdam

Today

1. Non-deterministic FSAs
2. Finite-state transducers (FSTs)
3. Word classes
4. Break
5. Syntax trees
6. Formal grammars (in particular, CFGs)

Last time: regular expressions

formal language :: a set of strings over an alphabet Σ ($\subseteq \Sigma^*$)

regular expressions :: a notation for characterising formal languages – declarative method

	Regular expression	Languages
empty string:	$\emptyset$	$\{\}$
any symbol $a \in \Sigma$:	a	$\{a\}$
If a and b are regexps, so are:		
concatenation:	$a \cdot b$	$\{ab\}$
disjunction (or union):	$(a\|b)$	$\{a, b\}$
Kleene star (or closure):	a^*	$\{\epsilon, a, aa, aaa, \ldots\}$

Last time: regular expressions

formal language :: a set of strings over an alphabet Σ ($\subseteq \Sigma^*$)

regular expressions :: a notation for characterising formal languages – declarative method

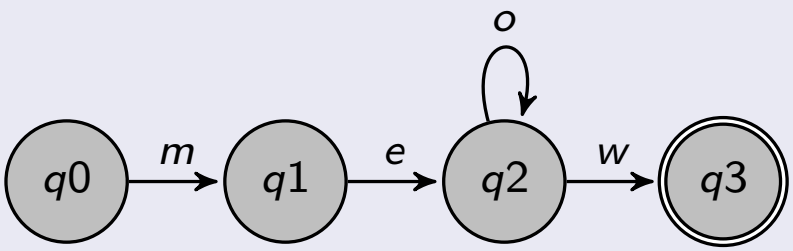
	Regular expression	Languages
empty set:	$\emptyset$	$\{\}$
empty string:	ϵ	$\{\epsilon\}$
any symbol $a \in \Sigma$:	a	$\{a\}$
If a and b are regexps, so are:		
concatenation:	$a \cdot b$	$\{ab\}$
disjunction (or union):	$(a\|b)$	$\{a, b\}$
Kleene star (or closure):	a^*	$\{\epsilon, a, aa, aaa, \ldots\}$

Last time: regular expressions as FSAs

For each regular expression, there is an equivalent FSA.

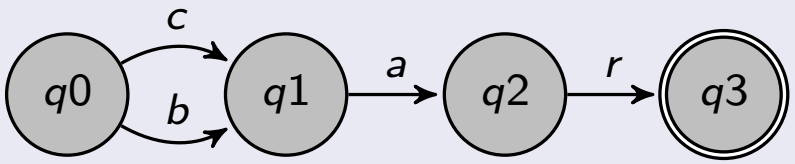
regular expression: meo^*w

language: $\{mew, meow, meoow, meooow...\}$



regular expression: $(b|c)ar$

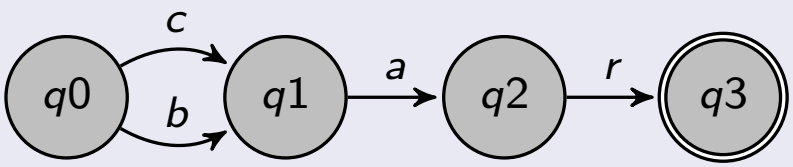
language: $\{bar, car\}$



Last time: formal definition of an FSA

We can formally specify an FSA by five parameters:

- Q a finite set of states
- Σ an input alphabet
- $q_0 \in Q$ the start state
- F a set of final or accepting states ($F \subseteq Q$)
- δ a transition function between states that maps pairs of states and input symbols $\langle q, i \rangle$ to a new state q' ($Q \times \Sigma \rightarrow Q$).



$Q = \{q_0, q_1, q_2, q_3\}$
$\Sigma = \{a, b, c, r\}$
start state: q_0
$F = \{q_3\}$
$\delta = \{(\langle q_0, c\rangle, q_1), (\langle q_0, b\rangle, q_1),$
$(\langle q_1, a\rangle, q_2), (\langle q_2, r\rangle, q_3)\}$

From reg exp to FSA: Kleene * or closure

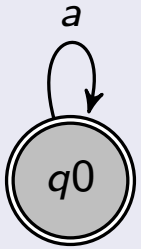
Regular expression:

- a^*: the concatenation of a with itself any number of times.

Corresponding FSA:

- Build FSA for a (A).
- Add a new final state and link all of A's final states to it with ϵ-transitions.
- Link A's initial state to the new final state with an ϵ-transition in both directions.

In the simplest case (a), after simplification, this amounts to:



Finite-state automata: some remarks

- There is only 1 start state...

Finite-state automata: some remarks

- There is only 1 start state...
- ...but there can be several final states.

Finite-state automata: some remarks

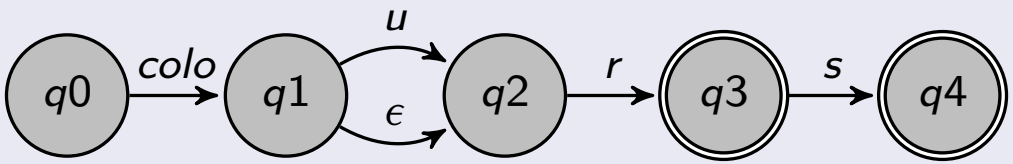
- There is only 1 start state...
- ...but there can be several final states.
- The start state and the final state may be the same.

Finite-state automata: some remarks

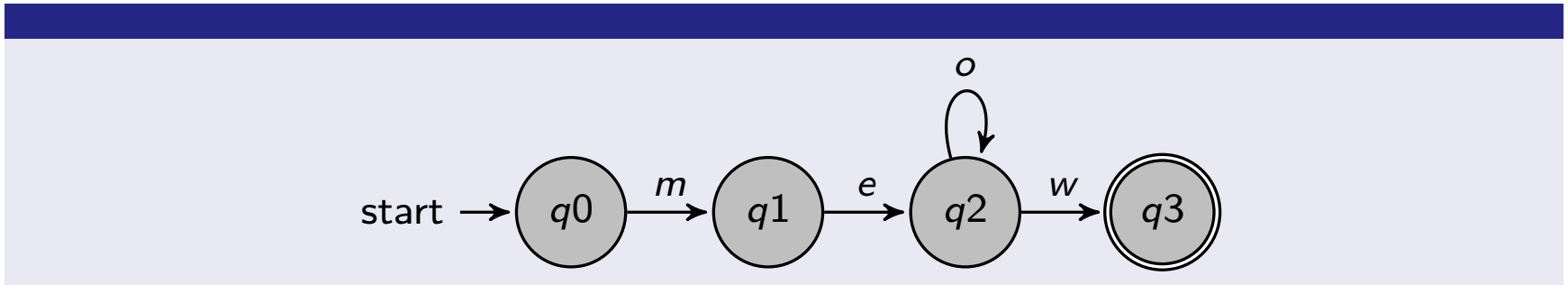
- There is only 1 start state...
- ...but there can be several final states.
- The start state and the final state may be the same.
- Every time we traverse the FSA from the start state to one of the final states, we have a string of the formal language (a *legal* string).

Finite-state automata: some remarks

- There is only 1 start state...
- ...but there can be several final states.
- The start state and the final state may be the same.
- Every time we traverse the FSA from the start state to one of the final states, we have a string of the formal language (a *legal* string).
- In case a final state has an outgoing arrow, we may continue as long as we can reach a final state later on.



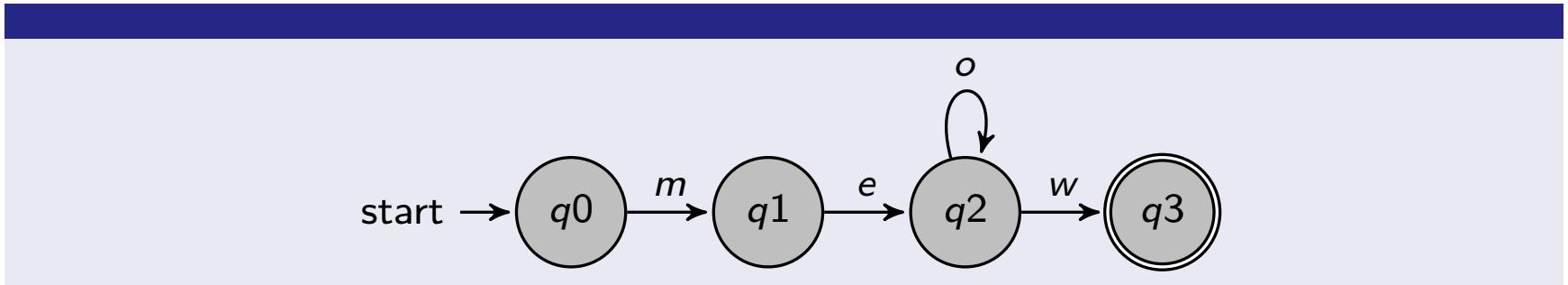
FSAs as generators and recognizers



FSAs can be used to generate strings:

- Each valid computation (from the start state to some final state) generates one of the possible strings of a language.

FSAs as generators and recognizers



FSAs can be used to generate strings:

- Each valid computation (from the start state to some final state) generates one of the possible strings of a language.

And they can also be used to recognize strings:

- Does a string belong to the language defined by the automaton?
- If the automaton consumes all input symbols in a string and reaches a final state, the string is accepted; else, it is rejected.

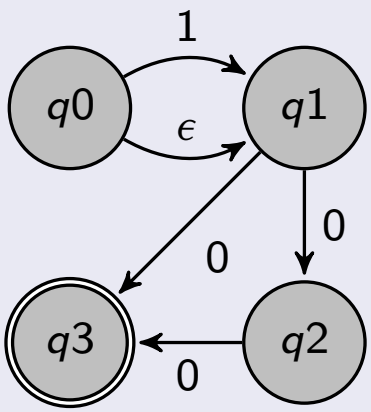
Non-deterministic FSAs (NFSA)

Deterministic FSAs

- Given a state and an input: the behavior of DFSA is fully deterministic.
  - For every state $q \in Q$ and symbol $a \in \Sigma$ there is exactly one state q' for which there exists a transition $q \xrightarrow{a} q'$ in δ.

Non-deterministic FSAs FSA is Non-deterministic if, for any state q and an input a, we can transition to more than one state.

Non-deterministic FSAa (NFSAs)

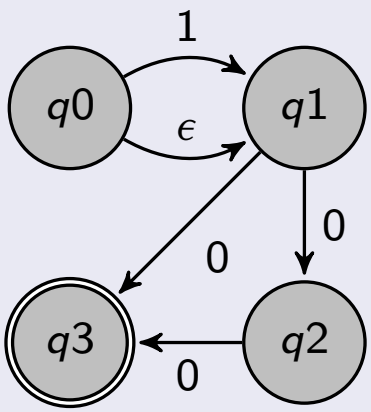


start: $q0$

$F = \{q3\}$

$$\delta = \{ \ (\langle q_0, 1\rangle, q_1), \\ (\langle q_0, \epsilon\rangle, q_1), \\ (\langle q_1, 0\rangle, q_2), \\ (\langle q_1, 0\rangle, q_3), \\ (\langle q_2, 0\rangle, q_3) \ \}$$

Non-deterministic FSAa (NFSAs)



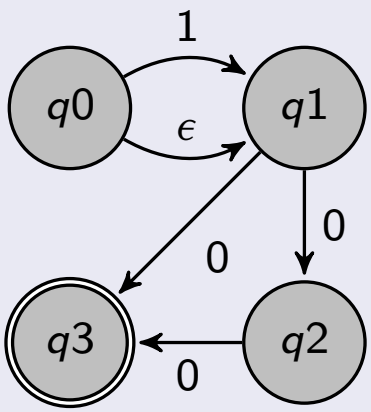
start: $q0$

$F = \{q3\}$

$$\delta = \{ \begin{array}{l} (\langle q_0, 1\rangle, q_1), \\ (\langle q_0, \epsilon\rangle, q_1), \\ (\langle q_1, 0\rangle, q_2), \\ (\langle q_1, 0\rangle, q_3), \\ (\langle q_2, 0\rangle, q_3) \ \} \end{array}$$

Recognizer: If we can transition to more than one state without conflicting with the input string, the FSA is non-deterministic.

Non-deterministic FSAa (NFSAs)



start: $q0$

$F = \{q3\}$

$$\delta = \{ \ (\langle q_0, 1\rangle, q_1), \\ (\langle q_0, \epsilon\rangle, q_1), \\ (\langle q_1, 0\rangle, q_2), \\ (\langle q_1, 0\rangle, q_3), \\ (\langle q_2, 0\rangle, q_3) \ \}$$

Recognizer: If we can transition to more than one state without conflicting with the input string, the FSA is non-deterministic.

Sources of non-determinism:

- states with more than one outgoing arrow with the same label (leading to different states)
- states with more than one outgoing arrow where some arrow is an ϵ-transition

Equivalence of NFSAs and DFSAs

- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).

Equivalence of NFSAs and DFSAs

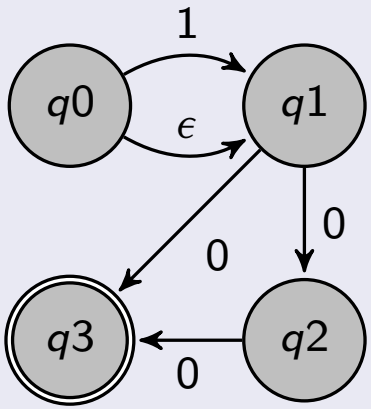
- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).
- Simplifying somewhat, to build a DFSA from an NFSA we need to create a deterministic path for each possible path in the NFSA.

Equivalence of NFSAs and DFSAs

- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).
- Simplifying somewhat, to build a DFSA from an NFSA we need to create a deterministic path for each possible path in the NFSA.

Equivalence of NFSAs and DFSAs

- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).
- Simplifying somewhat, to build a DFSA from an NFSA we need to create a deterministic path for each possible path in the NFSA.



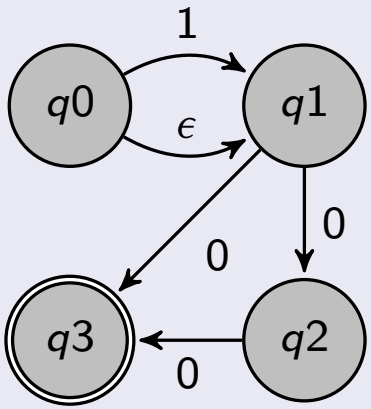
start: $q0$

$F = \{q3\}$

$$\delta = \{ \begin{array}{l} (\langle q_0, 1\rangle, q_1), \\ (\langle q_0, \epsilon\rangle, q_1), \\ (\langle q_1, 0\rangle, q_2), \\ (\langle q_1, 0\rangle, q_3), \\ (\langle q_2, 0\rangle, q_3) \ \} \end{array}$$

Equivalence of NFSAs and DFSAs

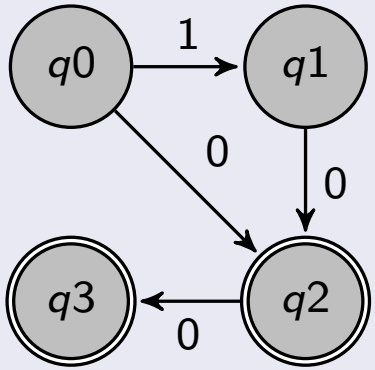
- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).
- Simplifying somewhat, to build a DFSA from an NFSA we need to create a deterministic path for each possible path in the NFSA.



start: $q0$

$F = \{q3\}$

$$\delta = \{ (\langle q_0, 1\rangle, q_1), (\langle q_0, \epsilon\rangle, q_1), (\langle q_1, 0\rangle, q_2), (\langle q_1, 0\rangle, q_3), (\langle q_2, 0\rangle, q_3) \}$$



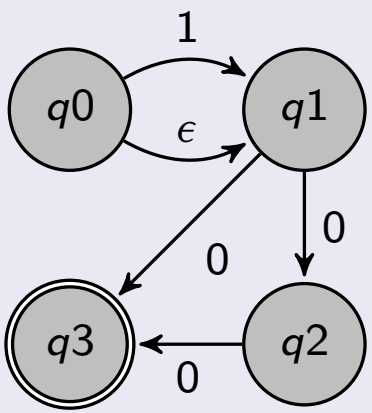
start: $q0$

$F = \{q2, q3\}$

$$\delta = \{ (\langle q_0, 1\rangle, q_1), (\langle q_0, 0\rangle, q_2), (\langle q_1, 0\rangle, q_2), (\langle q_2, 0\rangle, q_3) \}$$

Equivalence of NFSAs and DFSAs

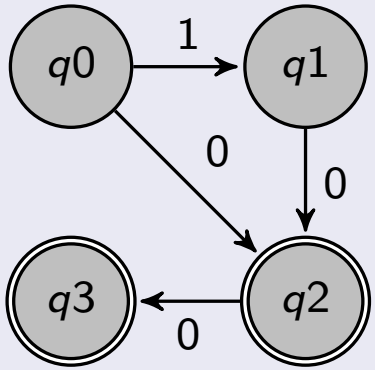
- For every NFSA, there is an equivalent DFSA – in the sense that both accept exactly the same set of strings (same language).
- Simplifying somewhat, to build a DFSA from an NFSA we need to create a deterministic path for each possible path in the NFSA.



start: $q0$

$F = \{q3\}$

$\delta = \{ (\langle q_0, 1\rangle, q_1),$
$(\langle q_0, \epsilon\rangle, q_1),$
$(\langle q_1, 0\rangle, q_2),$
$(\langle q_1, 0\rangle, q_3),$
$(\langle q_2, 0\rangle, q_3) \}$



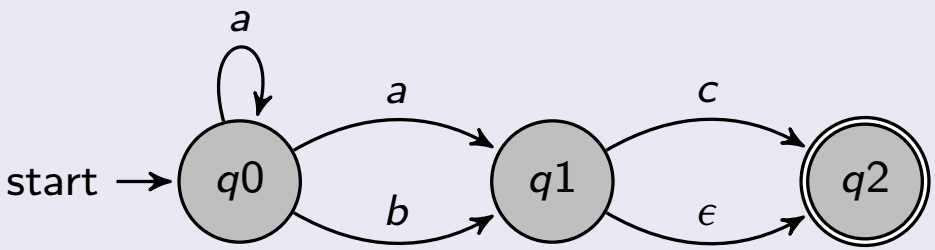
start: $q0$

$F = \{q2, q3\}$

$\delta = \{ (\langle q_0, 1\rangle, q_1),$
$(\langle q_0, 0\rangle, q_2),$
$(\langle q_1, 0\rangle, q_2),$
$(\langle q_2, 0\rangle, q_3) \}$

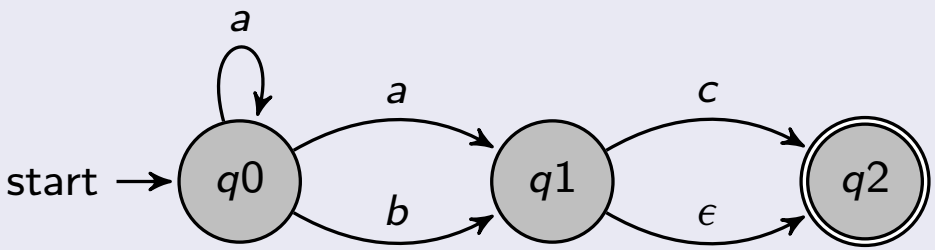
The recognition behavior of a DFSA is fully determined by the state it is in and the symbol it is processing.

Exercise



- What is the language generated by this FSA?
- Is it deterministic or non-deterministic?
- Why?
- Give an equivalent DFSA / FSA.

Exercise



- What is the language generated by this FSA?
- Is it deterministic or non-deterministic?
- Why?
- Give an equivalent DFSA / FSA.

NFSAs are not more powerful than FSAs, but they may be simpler.

Morphology

‘I goed fast!’

If a toddler runs up to you and announces *I go-ed fast!*, you will naturally respond, *Yes, you went fast!*

‘I goed fast!’

If a toddler runs up to you and announces *I go-ed fast!*, you will naturally respond, *Yes, you went fast!*

- You are correcting an error in morphology: the past tense of *go* isn't *goed* but *went*.

‘I goed fast!’

If a toddler runs up to you and announces *I go-ed fast!*, you will naturally respond, *Yes, you went fast!*

- You are correcting an error in morphology: the past tense of *go* isn't *goed* but *went*.
- The error demonstrates that the child understands some syntax, viz., how to combine personal pronouns, verbs of movement, and that English verbs have a simple past tense.

‘I goed fast!’

How can we make a computer understand morphology and syntax?

‘I goed fast!’

How can we make a computer understand morphology and syntax?

- Finite-state transducers (FSTs) extend FSAs to support morphological transformations.
- Formal grammars allow a computer to organize sentences of classified words efficiently into syntax trees.
- Part-of-speech tagging (not much in this course) assigns word classes to bridge the space between morphology and syntax.

The lexicon

The lexicon

A lexicon is a repository of words ($\approx$ a dictionary).

The lexicon

A lexicon is a repository of words (≈ a dictionary).

- What sort of information about words should a lexicon contain?
- How can it be represented efficiently?

The lexicon

A lexicon is a repository of words (≈ a dictionary).

- What sort of information about words should a lexicon contain?
- How can it be represented efficiently?

Words are made up of morphemes, the smallest meaningful units in a (natural) language.

category	lemma	inflectional affixes	derivational affixes
V	walk	-s, -ing, -ed	-able, -er
N	health	-s,	-y, un-

The lexicon

A lexicon is a repository of words (≈ a dictionary).

- What sort of information about words should a lexicon contain?
- How can it be represented efficiently?

Words are made up of morphemes, the smallest meaningful units in a (natural) language.

category	lemma	inflectional affixes	derivational affixes
V	walk	-s, -ing, -ed	-able, -er
N	health	-s,	-y, un-

Dictionaries typically don't include all the inflected forms of a word.

The lexicon

A lexicon is a repository of words (≈ a dictionary).

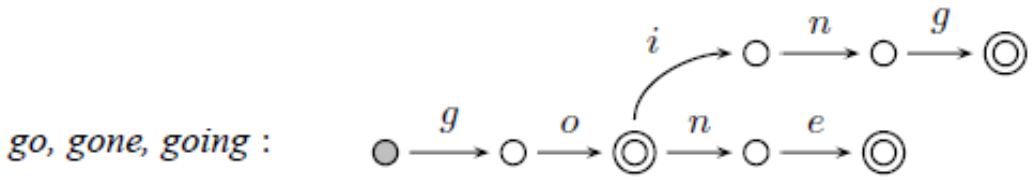
- What sort of information about words should a lexicon contain?
- How can it be represented efficiently?

Words are made up of morphemes, the smallest meaningful units in a (natural) language.

category	lemma	inflectional affixes	derivational affixes
V	walk	-s, -ing, -ed	-able, -er
N	health	-s,	-y, un-

Dictionaries typically don't include all the inflected forms of a word.

We can use an FSM to define the inflectional paradigm of a word:



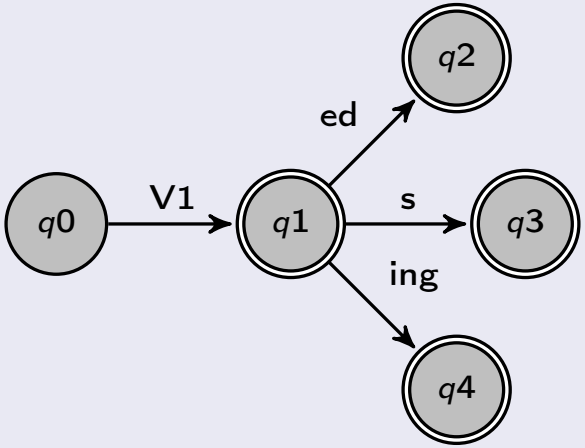
Morphological lexicons

An efficient way to define morphological lexicons consists in specifying FSAs that apply to sub-categories of words.

Morphological lexicons

An efficient way to define morphological lexicons consists in specifying FSAs that apply to sub-categories of words.

category	**lemma**
V1	walk, talk
V2	go, do



This accounts for the productivity of morphological processes (if we learn a new verb, we can guess its inflectional paradigm).

Morphological lexicons

A morphological lexicon lists the lemmas and other morphemes in a language (and their categories) and specifies the allowable morpheme sequences that make up the words.

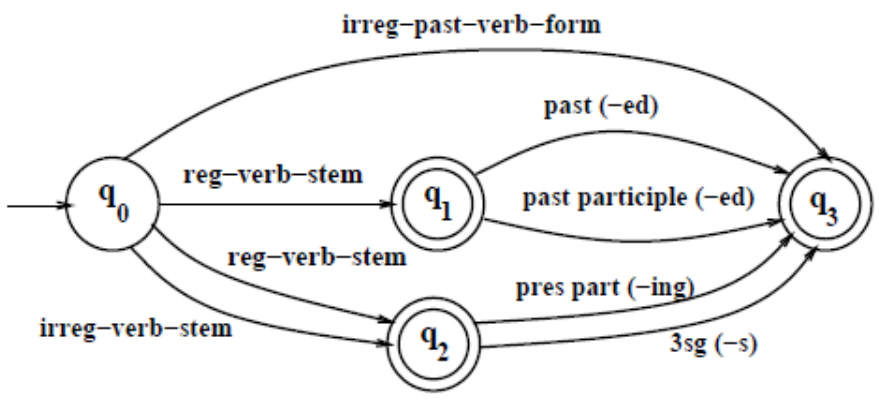


Figure 3.4 A finite-state automaton for English verbal inflection

reg-verb-stem	irreg-verb-stem	irreg-past-verb	past	past-part	pres-part	3sg
walk	cut	caught	-ed	-ed	-ing	-s
fry	speak	ate				
talk	sing	eaten				
impeach		sang				

Irregularities and transducers

Not all morphological processes consist of concatenating morphemes.

foot $\rightsquigarrow$ feet	make $\rightsquigarrow$ makes, making, made

Irregularities and transducers

Not all morphological processes consist of concatenating morphemes.

foot $\rightsquigarrow$ feet	make $\rightsquigarrow$ makes, making, made

To account for irregular forms, we require transformations:

$$
\begin{aligned}
f &\rightarrow f \\
o &\rightarrow e \\
o &\rightarrow e \\
t &\rightarrow t
\end{aligned}
$$

Irregularities and transducers

Not all morphological processes consist of concatenating morphemes.

foot $\rightsquigarrow$ feet	make $\rightsquigarrow$ makes, making, made

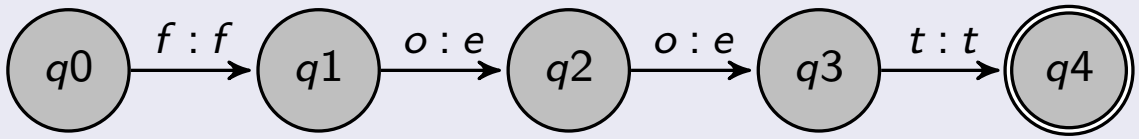
To account for irregular forms, we require transformations:

f $\rightarrow$ f
o $\rightarrow$ e
o $\rightarrow$ e
t $\rightarrow$ t

We can define such transformation with a variant of FSAs that maps between two sets of symbols: finite-state transducers (FSTs).

Finite-state transducers

- Q a finite set of states
- Σ an input alphabet
- Δ an output alphabet
- q_0 the start state
- $F \subset Q$, a set of final states
- δ a transition function that maps pairs of states and symbols $i \in \Sigma$ to pairs of states and symbols $j \in \Delta$ $(Q \times \Sigma \rightarrow Q \times \Delta)$



$Q = \{q_0, q_1, q_2, q_3, q_4\}$
$\Sigma = \Delta = \{a, b, c, d..., z\}$
start state: q_0
$F = \{q_4\}$

$$\delta = \{ (\langle q_0, f\rangle, \langle q_1, f\rangle), (\langle q_1, o\rangle, \langle q_2, e\rangle), (\langle q_2, o\rangle, \langle q_3, e\rangle), (\langle q_3, t\rangle, \langle q_4, t\rangle) \}$$

Lemmatisation

Lemmatisation

Lemmatisation is the process of reducing the forms in an inflexional paradigm of a word to their underlying common lemma.

Lemmatisation

Lemmatisation is the process of reducing the forms in an inflexional paradigm of a word to their underlying common lemma.

sing, sings, singing, sang, sung → sing

walk, walks, walking, walked → walk

Why is it important? In information retrieval applications like web search, when we search for keywords we want to find documents that contain any inflectional variants.

Lemmatisation

Lemmatisation is the process of reducing the forms in an inflexional paradigm of a word to their underlying common lemma.

sing, sings, singing, sang, sung → sing
walk, walks, walking, walked → walk

Why is it important? In information retrieval applications like web search, when we search for keywords we want to find documents that contain any inflectional variants.

We can use FSTs for both regular and irregular forms.

Transformation Types	
$x : x$	identity (no change)
$x : y$	substitution
$x : \epsilon$	deletion
$\epsilon : x$	insertion

Stemming

Stemming has the same goal as lemmatization, but does the reduction with less knowledge, using heuristic rules.

- It does not rely on a morphological lexicon (lemmas are not known).
- It tries to leave only the 'stem' (an approximation of a lemma) by stripping off the endings of the words.

ATIONAL $\rightarrow$ ATE (e.g., relational $\rightarrow$ relate)
ING $\rightarrow \epsilon$ if stem contains vowel (e.g., motoring $\rightarrow$ motor)

Stemming

Stemming has the same goal as lemmatization, but does the reduction with less knowledge, using heuristic rules.

- It does not rely on a morphological lexicon (lemmas are not known).
- It tries to leave only the 'stem' (an approximation of a lemma) by stripping off the endings of the words.

ATIONAL $\rightarrow$ ATE (e.g., relational $\rightarrow$ relate)
ING $\rightarrow \epsilon$ if stem contains vowel (e.g., motoring $\rightarrow$ motor)

The stemming rules can also be implemented as FSTs.

Stemming

Stemming has the same goal as lemmatization, but does the reduction with less knowledge, using heuristic rules.

- It does not rely on a morphological lexicon (lemmas are not known).
- It tries to leave only the 'stem' (an approximation of a lemma) by stripping off the endings of the words.

ATIONAL → ATE (e.g., relational → relate)
ING → ϵ if stem contains vowel (e.g., motoring → motor)

The stemming rules can also be implemented as FSTs.

Stemming can be useful, but can easily lead to mistakes.

national → **nate*

Morphological parsing

To construct well-formed sentences we need to pay attention to morphological features (e.g., agreement between subject and verb).

Morphological parsing

To construct well-formed sentences we need to pay attention to morphological features (e.g., agreement between subject and verb).

His foot *is* broken.	His feet *are* broken.

Morphological parsing

To construct well-formed sentences we need to pay attention to morphological features (e.g., agreement between subject and verb).

His foot *is* broken.	His feet *are* broken.

We would like to map the surface form (the words as they appear in a text) to more informative representations:

Morphological parsing

To construct well-formed sentences we need to pay attention to morphological features (e.g., agreement between subject and verb).

His foot *is* broken.	His feet *are* broken.

We would like to map the surface form (the words as they appear in a text) to more informative representations:

structured output:	foot -N-SG	foot -N-PL
	↑↑↑↑ ↑ ↑	↑↑↑↑ ↑ ↑
input form:	foot ϵ ϵ	feet ϵ ϵ

Parsing: producing some sort of linguistic structure for an input expression.

Morphological parsing

English		Spanish		
Input	Morphologically Parsed Output	Input	Morphologically Parsed Output	Gloss
cats	cat +N +PL	pavos	pavo +N +Masc +Pl	'ducks'
cat	cat +N +SG	pavo	pavo +N +Masc +Sg	'duck'
cities	city +N +Pl	bebo	beber +V +PInd +1P +Sg	'I drink'
geese	goose +N +Pl	canto	cantar +V +PInd +1P +Sg	'I sing'
goose	goose +N +Sg	canto	canto +N +Masc +Sg	'song'
goose	goose +V	puse	poner +V +Perf +1P +Sg	'I was able'
gooses	goose +V +1P +Sg	vino	venir +V +Perf +3P +Sg	'he/she came'
merging	merge +V +PresPart	vino	vino +N +Masc +Sg	'wine'
caught	catch +V +PastPart	lugar	lugar +N +Masc +Sg	'place'
caught	catch +V +Past			

Figure 3.2 Output of a morphological parse for some English and Spanish words. Spanish output modified from the Xerox XRCE finite-state language tools.

You can find several examples of FSTs for morphological parsing in the book (chapter 3).

FSTs: quiz time

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

Summary: morphology with finite-state transducers

- Morphology deals with the processes that govern the internal structure of words.
- These processes can be defined with finite-state means.
- FSTs are a variant of FSAs that define mappings between regular languages.
- FSTs are very useful to define morphological processes, including lemmatization, stemming, and morphological parsing.

Syntax: from words to sentences

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

**US of criticisms policy Chomsky threats has because of death received his foreign.*

Chomsky has received death threats because of his criticisms of US foreign policy.

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

**US of criticisms policy Chomsky threats has because of death received his foreign.*
Chomsky has received death threats because of his criticisms of US foreign policy.

- Speakers have intuitions about what are well-formed sentences of a language.

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

**US of criticisms policy Chomsky threats has because of death received his foreign.*
Chomsky has received death threats because of his criticisms of US foreign policy.

- Speakers have intuitions about what are well-formed sentences of a language.

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

**US of criticisms policy Chomsky threats has because of death received his foreign.*
Chomsky has received death threats because of his criticisms of US foreign policy.

- Speakers have intuitions about what are well-formed sentences of a language.

Colourless green ideas sleep furiously. — Noam Chomsky

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

**US of criticisms policy Chomsky threats has because of death received his foreign.*
Chomsky has received death threats because of his criticisms of US foreign policy.

- Speakers have intuitions about what are well-formed sentences of a language.

Colourless green ideas sleep furiously. — Noam Chomsky

Syntax: from words to sentences

Syntax deals with the structural properties of sentences.

- Not all sequences of words count as sentences of a language.

**US of criticisms policy Chomsky threats has because of death received his foreign.*

Chomsky has received death threats because of his criticisms of US foreign policy.

- Speakers have intuitions about what are well-formed sentences of a language.

Colourless green ideas sleep furiously. — Noam Chomsky

The gostak distims the doshes. — Andrew Ingraham

Word classes: parts of speech

Linguists have been classifying words for a long time.

- Dionysius Thrax of Alexandria (ca. 100 BC) wrote a grammatical sketch of Greek involving eight still-familiar parts of speech.

nouns	verbs	pronouns	prepositions
adverbs	conjunctions	participles	articles

Word classes: parts of speech

Linguists have been classifying words for a long time.

- Dionysius Thrax of Alexandria (ca. 100 BC) wrote a grammatical sketch of Greek involving eight still-familiar parts of speech.
- Thrax's list and minor variations of it have dominated European language grammars and dictionaries for 2000 years.

nouns	verbs	pronouns	prepositions
adverbs	conjunctions	participles	articles

Criteria for classifying words

When should words be put into the same class?

- Three criteria may be considered.

Distributional criteria
: Where can the word occur?

Morphological (or formal) criteria
: What form does the word have? What affixes can it take?

Notional (or semantic) criteria
: What sort of concept does the word refer to?

Open and closed classes

Open and closed classes

Open classes are typically large and have fluid membership.

Open and closed classes

Open classes are typically large and have fluid membership.

- Four major open-word classes are widely found in languages worldwide.
  - nouns
  - verbs
  - adjectives
  - adverbs

Open and closed classes

Open classes are typically large and have fluid membership.

- Four major open-word classes are widely found in languages worldwide.
  - nouns
  - verbs
  - adjectives
  - adverbs

Closed classes are typically small with relatively fixed membership.

- For example, determiners (*a*, *the*), prepositions (in English or Dutch), and postpositions (in Korean or Hindi).

Open and closed classes

Open classes are typically large and have fluid membership.

- Four major open-word classes are widely found in languages worldwide.
  - nouns
  - verbs
  - adjectives
  - adverbs

Closed classes are typically small with relatively fixed membership.

- For example, determiners (*a*, *the*), prepositions (in English or Dutch), and postpositions (in Korean or Hindi).
- Closed-class words (e.g., *of*, *which*, *could*) often play a structural role in the grammar as function words.

Nouns

Notionally, nouns refer to things.

- living things (*cat*)
- places (*Amsterdam*)
- nonliving things (*ship*)
- concepts (*marriage*, *algorithm*)

Nouns

Notionally, nouns refer to things.

- living things (*cat*)
- places (*Amsterdam*)
- nonliving things (*ship*)
- concepts (*marriage*, *algorithm*)

Formally, *-ness, -tion, -ity, -ance* tend to indicate nouns.

- *happiness*, *preparation*, *activity*, *significance*

Nouns

Notionally, nouns refer to things.

- living things (*cat*)
- places (*Amsterdam*)
- nonliving things (*ship*)
- concepts (*marriage*, *algorithm*)

Formally, *-ness, -tion, -ity, -ance* tend to indicate nouns.

- *happiness*, *preparation*, *activity*, *significance*

Distributionally, we can examine the contexts in which nouns occur.

- For example, nouns can appear with possession: *his car*, *her idea*, etc.

Verbs

Notionally, verbs refer to actions.

- *write*, *think*, *observe*

Verbs

Notionally, verbs refer to actions.

- *write*, *think*, *observe*

Formally, words that end in *-ate* or *-ise/-ize* tend to be verbs; words ending in *-ing* are often the present participle of a verb.

- *automate*, *calibrate*, *modernise*
- *washing*, *sleeping*

Verbs

Notionally, verbs refer to actions.

- *write*, *think*, *observe*

Formally, words that end in *-ate* or *-ise/-ize* tend to be verbs; words ending in *-ing* are often the present participle of a verb.

- *automate*, *calibrate*, *modernise*
- *washing*, *sleeping*

Distributionally, we can examine the contexts where a verb appears.

- Different types of verbs have different distributional properties. For example, base-form verbs can appear as infinitives: *to jump*, *to learn*.

Adjectives

Notionally, adjectives convey properties of or opinions about things that are nouns.

- *small*, *sensible*, *excellent*

Adjectives

Notionally, adjectives convey properties of or opinions about things that are nouns.

- *small*, *sensible*, *excellent*

Formally, words that end in *-al*, *-ble*, and *-ous* tend to be adjectives.

- *formal*, *gradual*, *sensible*, *generous*

Adjectives

Notionally, adjectives convey properties of or opinions about things that are nouns.

- *small*, *sensible*, *excellent*

Formally, words that end in *-al*, *-ble*, and *-ous* tend to be adjectives.

- *formal*, *gradual*, *sensible*, *generous*

Distributionally, adjectives usually appear before a noun or after some form of *to be*, e.g., *the big building*, or *John is tall*.

Adverbs

Notionally, adverbs convey properties of actions or events or adjectives.

- *quickly*, *often*, *possibly*, *unfortunately*
- *really*

Adverbs

Notionally, adverbs convey properties of actions or events or adjectives.

- *quickly*, *often*, *possibly*, *unfortunately*
- *really*

Formally, words that end in *-ly* tend to be adverbs.

Adverbs

Notionally, adverbs convey properties of actions or events or adjectives.

- *quickly*, *often*, *possibly*, *unfortunately*
- *really*

Formally, words that end in *-ly* tend to be adverbs.

Distributionally, adverbs can appear next to a verb or an adjective, or at the start of a sentence.

Formal and distributional criteria in practice

I really wish Mandriva would redesign the diskdrak UI. The orphan bit is borked.

- Often in reading, we come across unknown words.

Formal and distributional criteria in practice

I really wish Mandriva would redesign the diskdrak UI. The orphan bit is borked.

- Often in reading, we come across unknown words.
- Formal and distributional criteria help people (and machines) recognize which (open) class an unknown word belongs to.

Example: part-of-speech inference

Those zorls you splarded were margy.

What is the part-of-speech of *margy*?

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

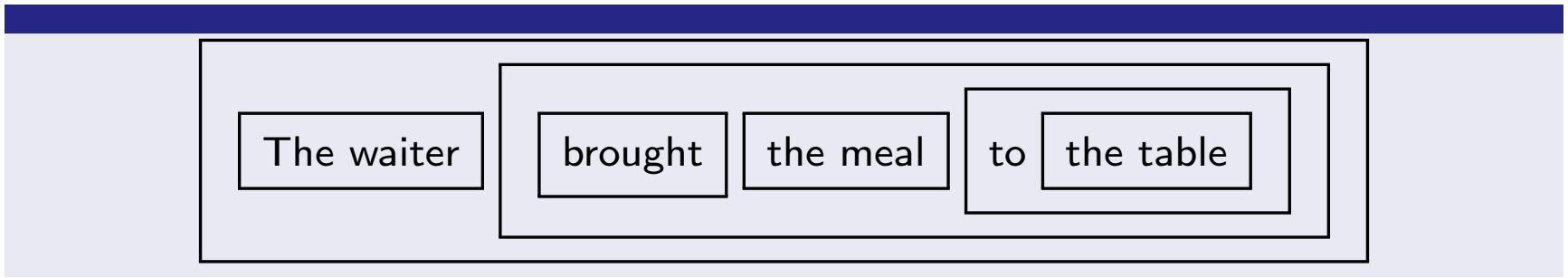
Larger pieces of text: constituents

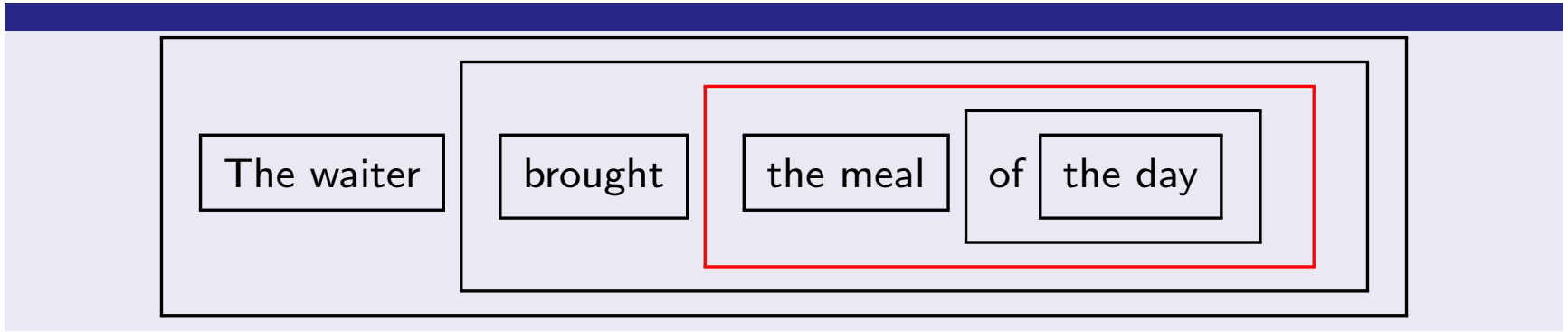
Larger pieces of text: constituents

In sentences, some groups of words fit together and act as units. We call these units constituents or phrases. They are named after their headword.

Larger pieces of text: constituents

In sentences, some groups of words fit together and act as units. We call these units constituents or phrases. They are named after their headword.

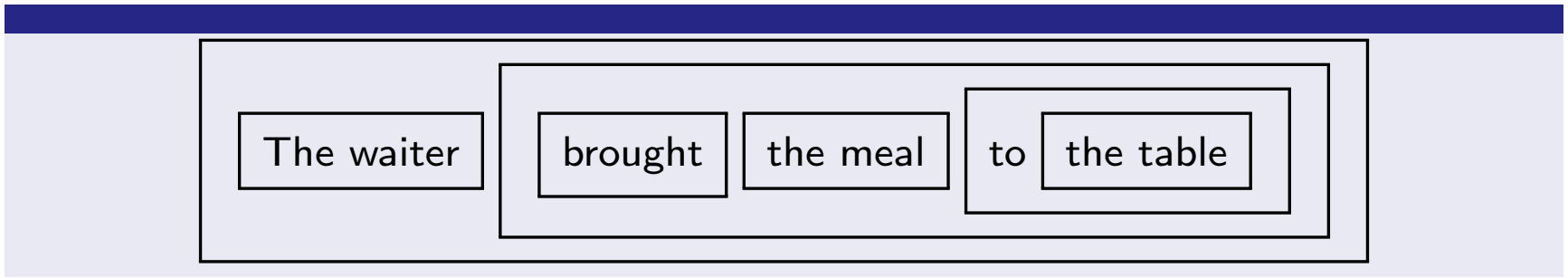


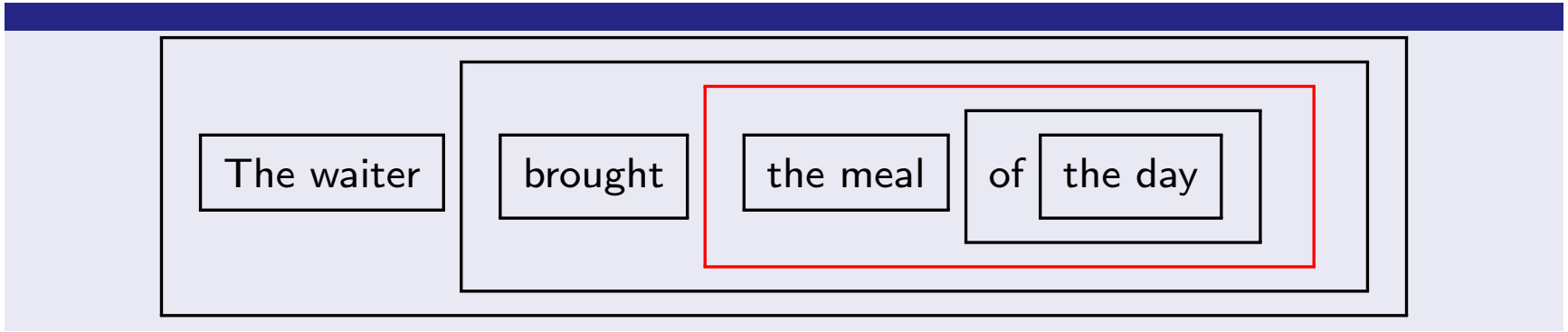


Noun Phrase (NP): The head is a noun, a proper name, or a pronoun.

Larger pieces of text: constituents

In sentences, some groups of words fit together and act as units. We call these units constituents or phrases. They are named after their headword.



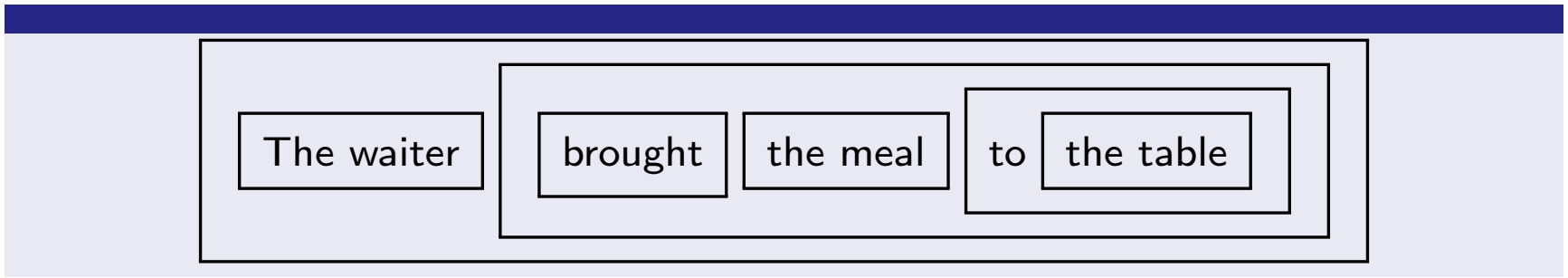


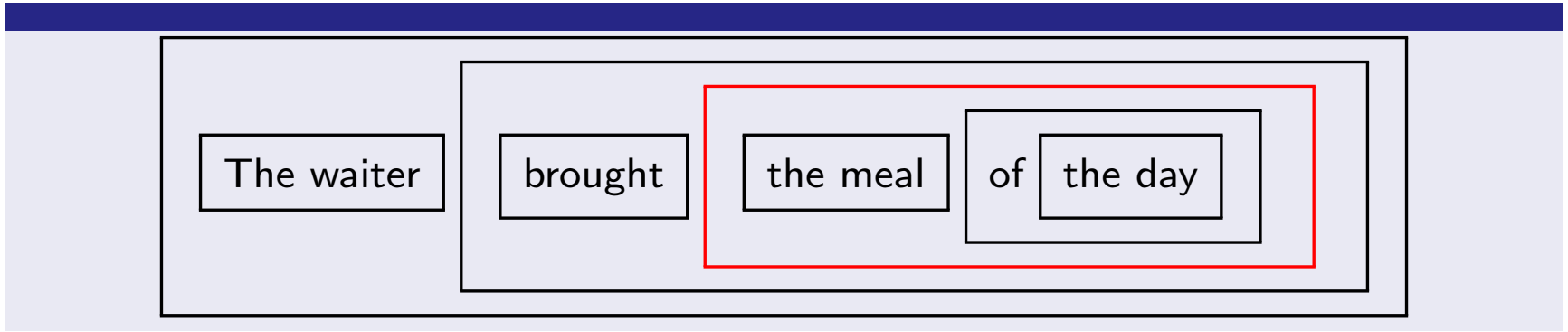
Noun Phrase (NP): The head is a noun, a proper name, or a pronoun.

Prepositional Phrase (PP): The head is a preposition followed by an NP.

Larger pieces of text: constituents

In sentences, some groups of words fit together and act as units. We call these units constituents or phrases. They are named after their headword.





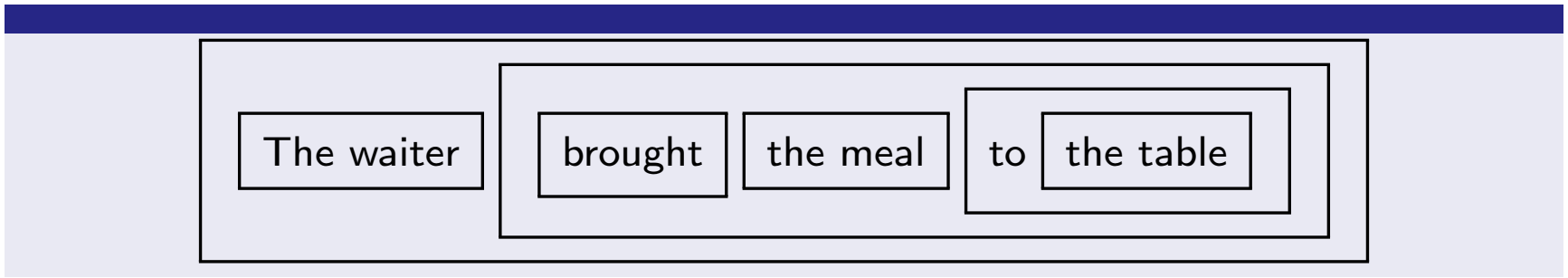
Noun Phrase (NP): The head is a noun, a proper name, or a pronoun.

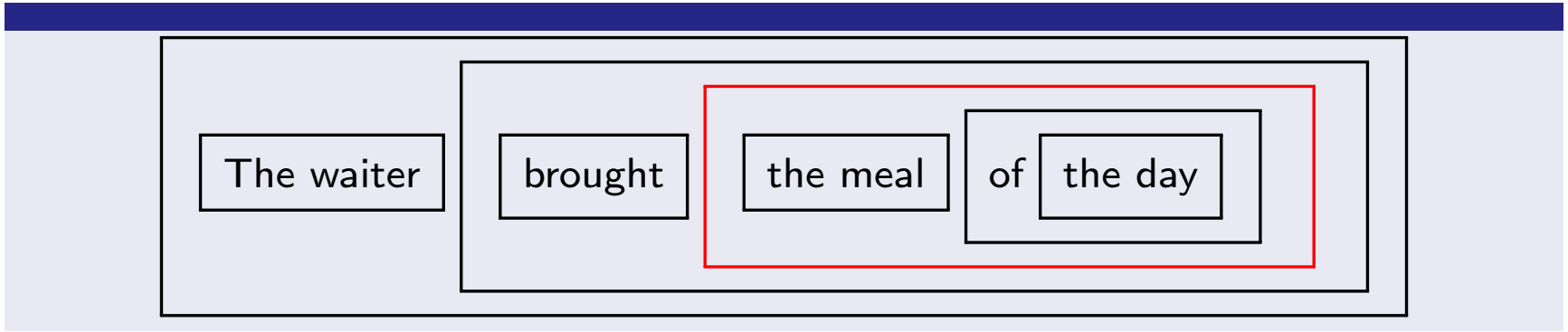
Prepositional Phrase (PP): The head is a preposition followed by an NP.

Verb Phrase (VP): The head is a verb and it includes its complements.

Larger pieces of text: constituents

In sentences, some groups of words fit together and act as units. We call these units constituents or phrases. They are named after their headword.





Noun Phrase (NP): The head is a noun, a proper name, or a pronoun.

Prepositional Phrase (PP): The head is a preposition followed by an NP.

Verb Phrase (VP): The head is a verb and it includes its complements.

Adjective Phrase (AP): The head is an adjective.

Constituency: basic idea

The criteria for defining constituents are sometimes fuzzy. The main criterion is distributional: words that are part of a constituent appear in similar positions in a sentence.

NPs	
Maria	*arrived*
three police cars	
they	
the chairman of the committee	
a dog that had been missing for days	

APs		
the	fast highly stylish most amazing red convertible	*car*

Tests for constituency

Tests for constituency

- Substitution test

Tests for constituency

- Substitution test

  The little boy fed the cat. → He fed her.

Tests for constituency

- Substitution test

  The little boy fed the cat. → He fed her.

- It-cleft test

  It is the meal that the waiter brought to the table.
  It is the meal of the day that the waiter brought to the table.
  *It is the meal to the table that the waiter brought.

Tests for constituency

- Substitution test

  The little boy fed the cat. → He fed her.

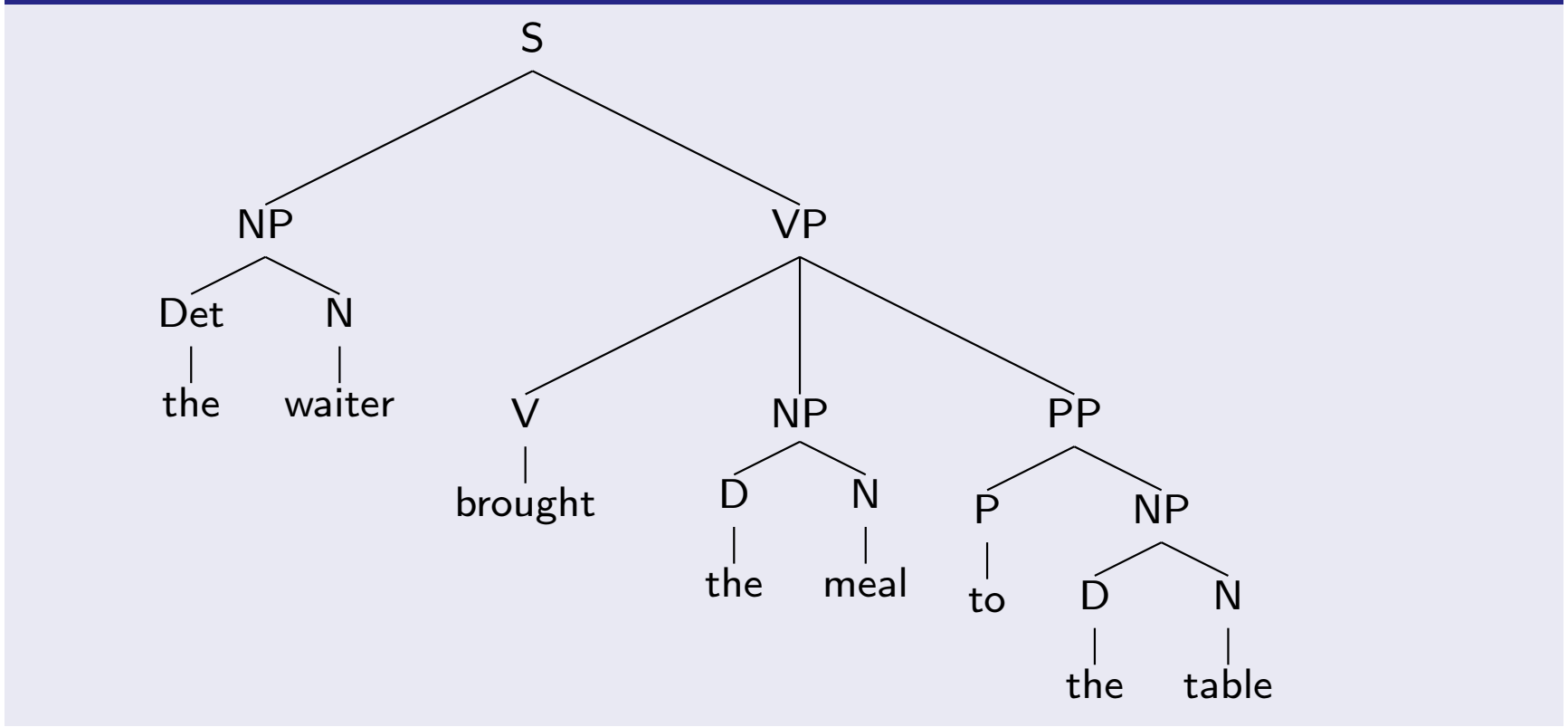
- It-cleft test

  It is the meal that the waiter brought to the table.
  It is the meal of the day that the waiter brought to the table.
  *It is the meal to the table that the waiter brought.

There are many linguistic theories of syntax, and it is not always easy or obvious to decide what constitutes a phrase.

Tree structures

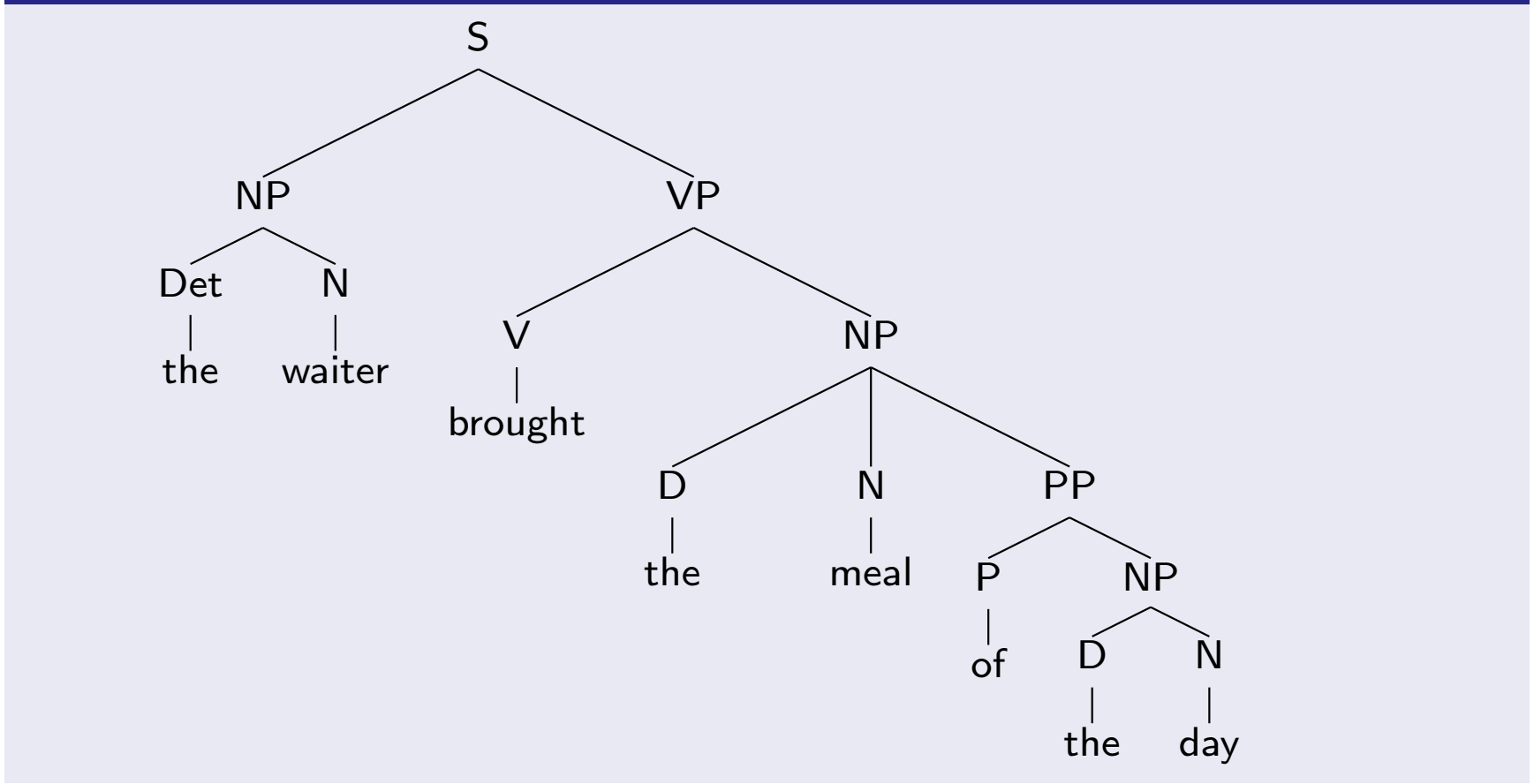
We can represent the phrase structure of sentences with syntactic trees.



- Each node in the tree represents a constituent.
- The leaves of the tree correspond to the words in the sentences.
- The symbol S corresponds to the top node, which denotes the whole sentence

Tree structures

We can represent the phrase structure of sentences with syntactic trees.



Formal grammars for syntactic analysis

The tree structures above can be modeled with phrase structure rules.

Formal grammars for syntactic analysis

The tree structures above can be modeled with phrase structure rules.

S → NP VP		(a sentence consists of an NP followed by a VP)
NP → D N		(an NP consists of a determiner followed by a noun)
VP → V NP		(a VP consists of a verb followed by an NP)
VP → V NP PP		(a VP may also consist of a verb followed by an NP and a PP)

etc.

Formal grammars for syntactic analysis

The tree structures above can be modeled with phrase structure rules.

S → NP VP	(a sentence consists of an NP followed by a VP)
NP → D N	(an NP consists of a determiner followed by a noun)
VP → V NP	(a VP consists of a verb followed by an NP)
VP → V NP PP	(a VP may also consist of a verb followed by an NP and a PP)

etc.

A collection of phrase structure rules of this sort constitutes a formal grammar for a particular language.

Regular expressions, FSAs, and FSTs are other formal devices to specify languages.

Ways to define a formal language

How can we formally **define or specify** the language(s) we are interested in?

1. Direct mathematical definition: $\Sigma = \{a, b, c\}$

$$L_1 = \{aa, bb, abc, abcc\}$$
$$L_2 = \{a^n b^n | n > 0\}$$

Ways to define a formal language

How can we formally **define or specify** the language(s) we are interested in?

1. Direct mathematical definition: $\Sigma = \{a, b, c\}$

$$L_1 = \{aa, bb, abc, abcc\}$$
$$L_2 = \{a^n b^n | n > 0\}$$

2. Formalisms (formal expressions and grammars): sets of rules
   - *Declarative method*

Ways to define a formal language

How can we formally **define or specify** the language(s) we are interested in?

1. Direct mathematical definition: $\Sigma = \{a, b, c\}$

$$L_1 = \{aa, bb, abc, abcc\}$$
$$L_2 = \{a^n b^n | n > 0\}$$

2. Formalisms (formal expressions and grammars): sets of rules
   - *Declarative method*

3. Automata: computational devices for computing languages
   - Specify some machine for testing whether a string is legal
   - *Procedural method*

Formal grammars

Formally, a grammar can be specified by 4 parameters.

- Σ a finite set of terminal symbols
- N a finite set of non-terminal symbols
- S a special symbol $S \in N$ called the start symbol
- R a finite set of rules or productions containing:
  - a sequence of terminal or non-terminal symbols
  - the symbol '$\rightarrow$'
  - another sequence of terminal or non-terminal symbols.
  - Rules have the form: $\alpha \rightarrow \beta$, where $\alpha, \beta \in (N \cup \Sigma)^*$

Context-free grammar (CFG)

For now, we'll focus on a particular class of grammars, so-called context-free grammars, whose rules have a more restricted form.

- Σ a finite set of terminal symbols
- N a finite set of non-terminal symbols
- S a special symbol $S \in N$ called the start symbol
- R a finite set of rules or productions of the form
  - $X \rightarrow \alpha$, where $X \in N$, $\alpha \in (N \cup \Sigma)^*$

Conventions

- Non-terminal symbols are represented with uppercase letters.
- Terminal symbols are represented with lowercase letters.
- The left-side symbol of the first rule is the start symbol.

Formal grammars for natural language syntax

Terminals: We take these to be words.

Non-terminals:
- phrase types (NP, VP, PP, ...)
- word classes (N, V, P, Prep, A, Adj, ...)

Start: *S* stands for 'sentence'.

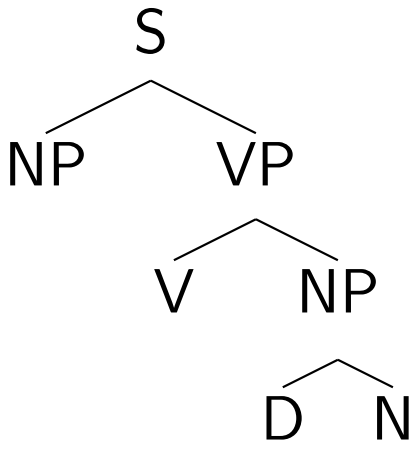
Rules:
- phrase structure rules (internal structure of phrases)
- lexicon (rules with a POS leading to a word)

Phrase structure rules:	Lexicon:
S → NP VP	N → waiter | meal | day | table
NP → D N	D → the
VP → V NP	V → brought
VP → V NP PP	P → of | to

Note that the same word may have two parts of speech: e.g., *love* can be N or V.

Why is this called context-free?

Context-free grammars are called context-free because a rule $X \rightarrow \alpha$ says that an X can always be expanded to α, no matter where the X occurs.



Generation, recognition, and parsing

A grammar can be used to generate or recognize sentences (sequences of terminals).

Generation: The formal language generated by a grammar is the set of sentences that can be derived by the application of grammar rules.

Recognition: If a sentence can be derived from the grammar rules, then it is syntactically well-formed, i.e., part of the language specified by the grammar or *grammatical*.

A grammar can also be used to parse sentence.

Parsing: The computational process of assigning a syntactic structure to a sentence is called parsing, which we will discuss next week.

Exercise: syntax trees

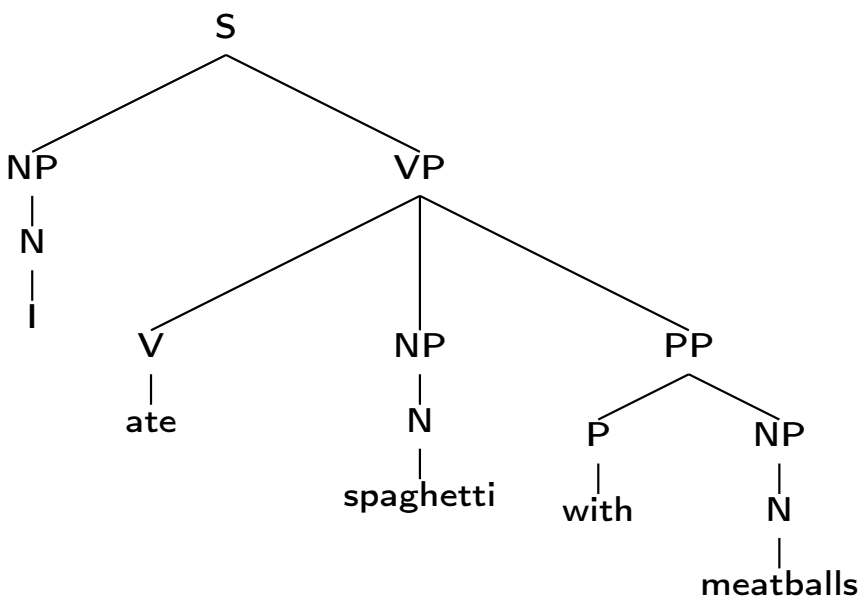
Phrase structure rules:	Lexicon:
$S \rightarrow NP\ \ VP$	$N \rightarrow I \mid spaghetti \mid chopsticks \mid meatballs$
$NP \rightarrow N$	$V \rightarrow ate$
$NP \rightarrow N\ PP$	$P \rightarrow with$
$VP \rightarrow V\ \ NP$	
$VP \rightarrow V\ \ NP\ \ PP$	
$PP \rightarrow P\ \ NP$	

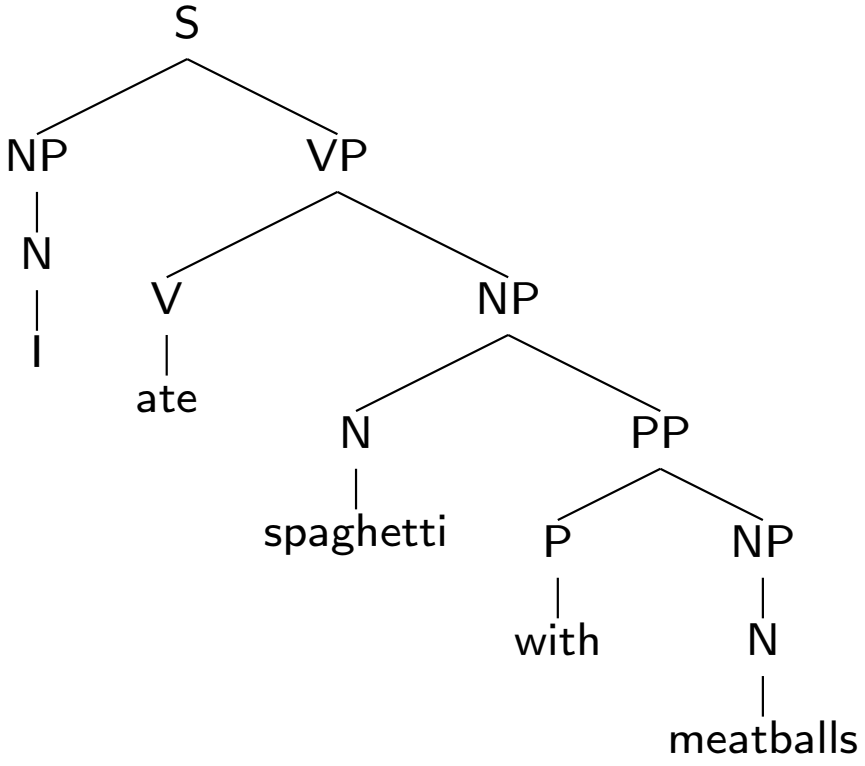
Given the (context-free) grammar above, draw syntactic trees for the following two sentences.

- *I ate spaghetti with meatballs.*
- *I ate spaghetti with chopsticks.*

Structural ambiguity

A grammar is structurally ambiguous if there is a string in its language that can be given two or more syntax trees.





Structural ambiguity: a poll

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
XAYUMJ

Enable answers by SMS

Recursion

Recursion in a grammar makes it possible to generate an infinite number of sentences.

Recursion

Recursion in a grammar makes it possible to generate an infinite number of sentences.

Direct recursion: A non-terminal on a rule's LHS also appears on its RHS.

$\text{VP} \rightarrow \text{VP Conj VP}$

$\text{Conj} \rightarrow \text{and} \mid \text{or}$

Recursion

Recursion in a grammar makes it possible to generate an infinite number of sentences.

Direct recursion: A non-terminal on a rule's LHS also appears on its RHS.

VP → VP Conj VP
Conj → and | or

Indirect recursion: Some non-terminal can be expanded (via several steps) to a sequence of symbols containing that non-terminal.

NP → Det N PP
PP → Prep NP

Summary: syntax with formal grammars

- Syntax deals with the structure of sentences.
- Syntactic constituents tell us how words combine to form units within a sentence.
- Grammars allow us to define what are well-formed sentences of a natural language and to assign internal structure to them.
- Within a grammar we can distinguish between the phrase structure rules (which tell us about constituents) and the lexicon (which classifies words according to their part of speech).

We can implement grammars easily in NLTK!